

Android applications are written in Java. However, the Android framework for executing applications, which is written in Java, does not take the traditional JVM/JRE approach. Instead, the Java code is compiled into byte-code (as in a pure JVM environment), which is then compiled into another Android-specific format known as the Dalvik EXecutable (DEX) format. This is run in an Android-specific virtual machine, known as the Dalvik VM, which is the heart of Android—and the biggest diversion from the pure Java-JVM approach. The Dalvik VM is highly optimised, both in size and execution efficiency. The different approach is because of three primary factors:

- **Size:** A *.dex* file is composed of several *.class* files; this gives an opportunity for duplicate data or code to be merged and shared among multiple *.class* files, which results in a smaller DEX file than possible with multiple *.class* files.
- **Run-time efficiency:** The Dalvik VM is highly optimised to execute DEX code—a major factor in the market's considerations of usable operating systems. We will later look at how this is achieved with 'zygotes'.
- **An extra layer of security:** DEX code is like the next level of abstraction over Java byte-code.

The Dalvik VM and the Androidised Linux kernel constitute the Android framework for applications, providing full separation between various applications—every application has its own unique instance of the Dalvik VM. This constitutes a very secure design. This means that for every application, Android starts a new instance of Dalvik. At first, this design may appear inefficient. However, a very important concept known as Zygote is primarily meant to address this problem. Zygote, which is always running in the background, is responsible for speeding up Dalvik instance creation as it has a pre-initialised footprint of Dalvik core components always ready in memory. Whenever a new instance of Dalvik needs to be created, Zygote speeds it up by using the pre-initialised core.

Android end-user applications and exposed libraries are written in Java. The application developer selects suitable Java classes that expose the required functionality or service.

The Native Development Kit (NDK)

Native code is written in C. So how can native code be integrated in the Android framework, which is Java dominant? Android has a solution for this—Native Development (ND). All code written in C (or in other words, all code that can directly run on Linux, outside Dalvik) is called 'native code'. You compile your native code through the NDK provided by Android, and generate a Linux shared library (not DEX format). This shared library can be used in Java applications through the Java Native Interface (JNI) mechanism. It does not run in the Dalvik VM, of course, but directly on the Linux kernel.

The Android security framework

From the developer's perspective, Android provides two levels of security to applications. Before discussing the details, let's look at how Android applications are shipped and installed on the device.

An Android application package is a file with the extension *.apk*. When a user installs such a file, the Android Package Manager (PM) identifies all the native libraries, along with the other components in the package, and puts these libraries in some package-specific location. Also, this application is assigned a unique UID (Linux user ID) and GID (group ID), based on the developer who signed the application package. From then onwards, this application is identified by the Linux kernel as a unique user—a one-app-one-user philosophy. Thus, all the separation provided to one user from another is now applicable to the Android application! Also, permissions to the location where all application data (including native libraries) resides, are as per the assigned UID/GID.

Now let's come back to the security framework.

Application sand-boxing

In Android, this term is used to describe the encapsulation provided to an application, with the help of Linux user-level (unique UID/GID) security. Each application runs in its own sandbox, and the Linux kernel maintains that separation, making every application's code and data accessible to that application only. Since this sand-boxing is directly by the kernel, it is very secure.

However, there is an exception to this rule: application developers can choose to share some data or native libraries between multiple applications. The restriction is that such applications should be created and signed by the same developer. In this case, more than one application (owned by the same vendor or developer) is assigned the same UID/GID by the Android package manager. This means that the default application sand-boxing can be 'broken' by the developers, making everything shared among their applications.

Let's look at how this model affects native code. Let us suppose an application is assigned a user ID *<X>*, and the application data storage location is *<PATH>*. Now, only *<PATH>* has the desired Linux permissions for the user ID *<X>*. So, if native code running as a part of this application tries to access anything at some other location, the Linux kernel will ensure that this access is denied!

Android permission framework

At the top level, which has the Android (Java) APIs, Android also has another security layer. Android defines a large set of security types (also known as capabilities), which are necessary for an application to exhibit in order to use a specific service from the Android runtime. For example, a permission with the name '*INTERNET*' is necessary if the application wants access to network-